

Multi-Process Video Encoding System

Project Report

Course: Operating Systems

Institution: FAST-NUCES

Students: Faraz Ahmed (24k0942) | Saad Aamer (23k0921) | Muhammed Talha
Aamir (24k0729)

1. Introduction

This report documents the design and implementation of the Multi-Process Video Encoding System, a project built for the Operating Systems course at FAST-NUCES. The system is a real, working video-encoding pipeline written in C (C11 standard) that demonstrates every major OS concurrency primitive taught in the course: processes, threads, mutexes, semaphores, and shared memory. Rather than a toy simulation, the code invokes FFmpeg under the hood to transcode actual video files and uses POSIX IPC mechanisms to coordinate work across multiple processes running in parallel on an Ubuntu system.

The report first explains what the system does in plain terms, then maps every component back to the OS concepts it exercises, discusses compliance with the project proposal, and flags areas where the implementation diverges from the original specification.

2. What the System Does

At a high level the system takes a single input video file and produces a re-encoded output video by splitting the encoding work across multiple worker processes that run at the same time. The steps are as follows:

- The user runs `./orchestrator input.mp4 [num_workers]`. The orchestrator is the main process and acts as the manager of the whole pipeline.
- The orchestrator uses `ffprobe` to count the total number of frames in the video and divides them into `N` roughly equal batches, one per worker.
- It creates `N` child processes (workers) using `fork()+execl()` and assigns each child a frame range to encode.
- Each worker runs `FFmpeg` to transcode its assigned frames into a short `.mp4` segment file. While `FFmpeg` is running, the worker also maintains four `pthreads` that simulate parallel frame-level processing and write progress into a shared output buffer protected by a `mutex`.
- When a worker finishes encoding its segment it posts a `POSIX` named semaphore to tell the orchestrator it is done and marks its status in shared memory.
- The orchestrator waits for all `N` semaphores (one per worker) and then uses `FFmpeg`'s `concat` demuxer to stitch all segments back into a single output video.
- A separate Python web dashboard (`gui_monitor.py`) reads the shared memory segment every 250 ms and displays live progress bars for every worker in a browser window.

3. System Architecture

The architecture has three executable components that communicate through the OS IPC layer. They never import each other as libraries; every piece of information crosses process boundaries via kernel-managed data structures.

3.1 Component Map

- `shared.h` -- A C header file included by both `orchestrator.c` and `worker.c`. It defines the `SharedMem` struct layout, the POSIX shared-memory name (`/vidpipe_shm`), the semaphore name prefix (`/vidpipe_sem_N`), and the `WorkerStatus` enum. This single source of truth prevents the two programs from reading each other's memory fields with different offsets.
- `orchestrator.c` -- The parent process. Creates shared memory, creates `N` named semaphores, forks `N` worker processes, monitors them with `waitpid()`, waits on each semaphore, runs the final FFmpeg merge, and cleans up all IPC resources.
- `worker.c` -- Each child process. Attaches to the existing shared memory, opens its own semaphore, spawns four pthreads, runs FFmpeg for its frame batch, and posts the semaphore when done.
- `gui_monitor.py` -- A standalone Python process. Attaches to the shared memory segment in read-only mode, parses it with Python's `struct` module, and serves a live HTML dashboard on `localhost:8080`.

3.2 Data Flow

The following sequence describes the data flow from the perspective of the OS:

- Orchestrator calls `shm_open(O_CREAT)` and `ftruncate()` to create a named shared memory object of `sizeof(SharedMem)` bytes in the kernel's tmpfs.
- Orchestrator calls `mmap(MAP_SHARED)` to map the object into its own virtual address space. It writes `num_workers` and `total_frames` into the struct.
- Orchestrator calls `fork()` `N` times. Each `fork()` duplicates the orchestrator's address space. The child immediately calls `execl('./worker', ...)` which replaces the child's address space with the worker binary, but the worker reconnects to the same named shared memory object by calling `shm_open()` itself (the name persists in the kernel until unlinked).
- Workers write into `worker_status[]` and `worker_frames[]` fields. Because `MAP_SHARED` is used every write is visible to every process that has the same object mapped.

- The Python GUI calls the librt `shm_open()` through `ctypes`, maps the same name, and reads the struct bytes with `struct.unpack()`.
- When a worker finishes, it calls `sem_post()` on its named semaphore. The orchestrator's `sem_timedwait()` call in `wait_all_semaphores()` unblocks for that worker.
- After all semaphores are collected the orchestrator calls `shm_unlink()` to delete the kernel object and `sem_unlink()` to destroy each semaphore.

4. Operating System Concepts Used

4.1 Processes and the fork-exec Pattern

A process is a running program instance with its own virtual address space, file descriptor table, and kernel-maintained PCB (Process Control Block). Creating a new process on Linux always goes through `fork()`, which creates an almost-identical copy of the calling process. The child and parent share the same code and data in copy-on-write pages; the kernel only copies a page when one of them actually writes to it.

In this project the orchestrator calls `fork()` in a loop inside `spawn_worker()`. The child branch immediately calls `execl('./worker', ...)` which uses the `execve()` system call to replace the child's entire address space with the worker binary. After `exec` the child is a brand-new worker process with its own stack and heap, but it retains its PID, open file descriptors, and any shared memory mappings inherited from before the `exec`. The orchestrator stores the returned PID in a `WorkerSlot` struct so it can track each child individually.

- `fork()` system call -- duplicates the calling process. Returns 0 in the child, child PID in the parent.
- `execl()` system call -- loads a new binary into the current process. Does not return on success.
- `waitpid(pid, &status, WNOHANG)` -- non-blocking check whether a specific child has exited. Used in the monitoring loop to detect crashes without blocking the orchestrator.
- `WIFEXITED(status) / WEXITSTATUS(status)` -- macros that decode the exit reason from the integer filled in by `waitpid`.

4.2 Threads and the POSIX Thread Library (pthreads)

A thread is a unit of execution that shares the virtual address space of its parent process. Creating a thread is much cheaper than forking a process because no memory copying is

needed and all global variables, heap allocations, and file descriptors are instantly visible to every thread in the process.

Each worker spawns `NUM_THREADS = 4` POSIX threads using `pthread_create()`. All four threads run the same function (`thread_worker`) and share the same global variables: `output_buffer[]`, `buffer_index`, `actual_frames_encoded`, and `current_frame_idx`. The worker design is a classic thread pool: threads loop, pick up the next available frame index under a lock, simulate encoding work, and write their result into the shared buffer.

- `pthread_create(&tid, NULL, thread_worker, NULL)` -- creates a new kernel-scheduled thread in the current process.
- `pthread_join(tid, NULL)` -- blocks the calling thread until the target thread exits. Used in `run_pthreads_and_encode()` to ensure all simulation threads complete before the worker posts its semaphore.
- `thread_worker(void*)` -- the thread entry point. The `(void)arg;` idiom suppresses the unused-parameter compiler warning without a cast.
- `actual_frames_encoded` -- a volatile int that the main thread updates while reading FFmpeg's progress output, and threads read to decide whether a frame is ready to simulate.

4.3 Mutual Exclusion with `pthread_mutex_t`

A mutex (mutual exclusion lock) is a synchronization primitive that allows at most one thread at a time to execute a critical section. If a second thread calls `pthread_mutex_lock()` while the mutex is held, it blocks in the kernel until the owner releases the lock with `pthread_mutex_unlock()`. This prevents data races.

A data race occurs when two threads access the same memory location concurrently and at least one access is a write. Without a mutex, two threads could both read `buffer_index = 5`, both write their frame into `output_buffer[5]`, and both increment `buffer_index` to 6, losing one frame entirely. This is a classic lost-update bug.

`worker.c` uses two statically initialized mutexes:

- `buffer_mutex` -- guards `output_buffer[]` and `buffer_index`. Acquired before writing a completed frame into the buffer and released immediately after. The critical section is deliberately small so that threads do not block each other during the simulated encoding work (the `nanosleep` call is outside the lock).
- `work_mutex` -- guards `current_frame_idx` and the comparison against `actual_frames_encoded`. This ensures that two threads cannot both claim the same frame index.

Both mutexes are declared with the `PTHREAD_MUTEX_INITIALIZER` macro, which is the static initializer equivalent of calling `pthread_mutex_init()`. Both are process-local: they live in the worker's data segment and are not shared across processes.

4.4 POSIX Named Semaphores

A semaphore is a non-negative integer counter managed by the kernel. Two atomic operations are defined on it: `sem_post()` (V operation) increments the counter, and `sem_wait()` (P operation) decrements it, blocking if the counter is zero. Unlike a mutex, a semaphore can be posted by a different process than the one that will wait on it, which makes it ideal for signaling completion across process boundaries.

Named semaphores live in the kernel's semaphore namespace (accessible as `/dev/shm` entries on Linux) and can be opened by name from any process that knows the name, just like a file.

The orchestrator creates `N` semaphores with initial value 0 in `create_semaphores()`:

```
sem_open("/vidpipe_sem_0", O_CREAT|O_EXCL, 0666, 0);
sem_open("/vidpipe_sem_1", O_CREAT|O_EXCL, 0666, 0);
... up to /vidpipe_sem_(N-1)
```

Because the initial value is 0, any call to `sem_wait()` on these semaphores will block immediately. Each worker opens its own semaphore by name and calls `sem_post()` exactly once when it finishes encoding:

```
sem_post(done_sem); /* unblocks the orchestrator */
```

The orchestrator calls `sem_timedwait()` with a 60-second absolute deadline. If a worker never posts (e.g., hangs indefinitely), the orchestrator will time out rather than deadlock. After collecting all semaphores the orchestrator calls `sem_unlink()` to remove them from the kernel namespace.

4.5 POSIX Shared Memory

Ordinary process memory is private: one process cannot read or write another process's virtual address space. Shared memory is the highest-bandwidth IPC mechanism available because once the mapping is established, accesses to the shared region are just normal load/store instructions with no system call overhead per access.

POSIX shared memory works by creating a named object in the kernel (backed by `tmpfs` on Linux) and then mapping it into each interested process's virtual address space with `mmap(MAP_SHARED)`. Any write to the mapped region is immediately visible to all other processes that have the same region mapped.

The orchestrator creates and initializes the shared memory in `shm_create()`:

```
int fd = shm_open("/vidpipe_shm", O_CREAT|O_RDWR, 0666);
```

```
ftruncate(fd, sizeof(SharedMem));
SharedMem *shm = mmap(NULL, sizeof(SharedMem),
PROT_READ|PROT_WRITE, MAP_SHARED, fd, 0);
```

Workers attach to the existing object without O_CREAT in shm_attach():

```
int fd = shm_open("/vidpipe_shm", O_RDWR, 0666);
SharedMem *shm = mmap(NULL, sizeof(SharedMem),
PROT_READ|PROT_WRITE, MAP_SHARED, fd, 0);
```

The SharedMem struct stores:

- num_workers -- how many workers are active (read by GUI).
- total_frames -- total frames in the source video.
- pipeline_done -- set to 1 by the orchestrator after the merge step; the GUI polls this to know when to launch the media player.
- worker_frames[MAX_WORKERS] -- each worker writes its own slot with the count of frames it has processed so far. The GUI reads all slots to draw progress bars.
- worker_total[MAX_WORKERS] -- frames assigned per worker, written by orchestrator at startup.
- worker_status[MAX_WORKERS] -- WorkerStatus enum (IDLE / ENCODING / DONE / CRASHED). Written by each worker, read by orchestrator and GUI.

Fields accessed concurrently by multiple processes are marked volatile so the C compiler does not cache them in registers. No additional lock is needed for these integer fields because each slot is written by only one worker and read by others, and a single aligned 32-bit write is atomic on all x86 platforms.

4.6 Process Monitoring and Error Recovery

The orchestrator's monitor_workers() function implements a polling supervisor loop. It calls waitpid(pid, &status, WNOHANG) for each live worker on every iteration. The WNOHANG flag prevents waitpid from blocking: if the child is still running it returns 0 immediately instead of sleeping.

If a worker exits with a non-zero status (crash) and has not exceeded RESPAWN_MAX = 3 attempts, the orchestrator:

- Logs the crash with the signal number (WTERMSIG) or exit code (WEXITSTATUS).
- Resets worker_frames[id] in shared memory to 0 so the GUI does not show stale data.
- Calls spawn_worker() again with the same frame range to create a replacement process.

After each full scan the loop sleeps for 100 ms using `nanosleep()` to avoid burning CPU in a tight busy-wait. This is the correct pattern for a supervisor: short sleeps rather than busy-waiting.

4.7 Signal Handling

The orchestrator registers a signal handler for `SIGINT`, `SIGTERM`, and `SIGTSTP` using `signal()`. If the user presses `Ctrl+C` or sends a kill signal, `handle_sigint()` is called by the kernel before the process exits. The handler calls `shm_destroy()` which unmaps the shared memory and unlinks the kernel object. Without this cleanup, stale shared memory objects would persist in `/dev/shm` between runs and cause name conflicts.

The Python GUI registers a handler for `SIGTSTP` (`Ctrl+Z`) using `signal.signal()` to call `os._exit(0)`, ensuring the HTTP server shuts down cleanly on that signal.

5. File-by-File Code Walkthrough

5.1 `shared.h`

This header is the contract between the orchestrator and all worker processes. It defines:

- `MAX_WORKERS = 16` -- the compile-time upper bound on the worker count. Both programs allocate array slots based on this constant, guaranteeing identical struct layouts.
- `SHM_NAME = /vidpipe_shm` -- the POSIX shared-memory object name. Changing it in one file would silently break communication.
- `SEM_PREFIX = /vidpipe_sem_` -- combined with a worker ID to produce per-worker semaphore names such as `/vidpipe_sem_0`.
- `WorkerStatus` enum -- four integer codes: `WORKER_IDLE(0)`, `WORKER_ENCODING(1)`, `WORKER_DONE(2)`, `WORKER_CRASHED(3)`. Both programs use the same numeric values because they include the same header.
- `SharedMem` struct -- the exact binary layout of the shared-memory region. Both programs compute field offsets from the same struct definition.
- `shm_mark_complete_if_exists()` -- declared in `shared.h` and defined in `orchestrator.c`. It opens the shared memory by name, sets `pipeline_done = 1`, and closes it again. This function is designed to be safe to call even if the object no longer exists.

5.2 orchestrator.c

Key functions and their OS roles:

- `count_frames()` -- opens a pipe to `ffprobe` using `popen()` and reads a single integer from `stdout`. `popen()` is a `libc` wrapper around `fork()+exec()+pipe()`.
- `shm_create(nw)` -- creates the shared memory object, sizes it with `ftruncate()`, maps it with `mmap(MAP_SHARED)`, and zeroes it with `memset()`. Returns a pointer valid in the orchestrator's address space.
- `create_semaphores(nw)` -- loops 0..N-1 and calls `sem_open(O_CREAT|O_EXCL)` for each worker with initial value 0. `sem_unlink()` is called first to remove any stale object from a previous run.
- `spawn_worker(id, start, count, video)` -- calls `fork()` and, in the child, constructs the four argument strings and calls `execl('./worker')`. The parent returns the child PID.
- `monitor_workers(slots, nw, shm)` -- the supervisor loop. Iterates until all slots are marked done. Uses `waitpid(WNOHANG)` to check for exits, respawns on crash, sleeps 100ms between iterations.
- `wait_all_semaphores(nw)` -- for each worker index calls `sem_timedwait()` with a 60-second absolute deadline. After a successful wait it calls `sem_unlink()` to clean up.
- `merge_segments(nw)` -- writes a `concat.txt` file listing each segment, then calls `system()` to run `FFmpeg concat`. After merge it calls `shm_mark_complete_if_exists()` to set `pipeline_done = 1` for the GUI.
- `shm_destroy(shm)` -- calls `munmap()` then `shm_unlink()` to release the kernel object.
- `handle_sigint(sig)` -- the cleanup signal handler registered for `SIGINT`, `SIGTERM`, and `SIGTSTP`.

5.3 worker.c

Key functions and their OS roles:

- `shm_attach()` -- opens the named shared memory object (must already exist, no `O_CREAT`) and maps it into the worker's address space.
- `thread_worker(void*)` -- the POSIX thread function. Acquires `work_mutex` to claim the next frame index, checks whether `FFmpeg` has encoded that frame yet (`actual_frames_encoded`), releases the lock, sleeps 10ms to simulate encoding, then acquires `buffer_mutex` to write into `output_buffer[]` and update `shm->worker_frames[id]`.

- `run_pthreads_and_encode(id, start, count, video, shm)` -- the main logic. Sets globals for threads, spawns four pthreads, runs FFmpeg via `popen()` to read `-progress pipe:1` output, parses `frame=` lines to advance `actual_frames_encoded`, calls `pclose()`, sets `actual_frames_encoded = total` to release waiting threads, then joins all four threads.
- `main()` -- parses four CLI arguments, validates them, calls `shm_attach()`, opens the named semaphore, calls `run_pthreads_and_encode()`, sets `WORKER_DONE`, posts the semaphore, and calls `munmap()`.

5.4 gui_monitor.py

The Python monitor is a completely independent process. It uses ctypes to call the C library function `shm_open()` from `librt.so` because Python's standard library does not expose POSIX shared memory directly. Once it has a file descriptor it creates a read-only mmap object and calls `struct.unpack()` with a format string that matches the C struct layout:

```
STRUCT_FMT = f"=iii{MAX_WORKERS}i{MAX_WORKERS}i{MAX_WORKERS}i"
```

The `=` prefix means native byte order, no alignment padding. This matches the C compiler's layout for a struct of ints on x86-64.

The HTTP server (Python's built-in `HTTPServer`) serves a single HTML page on port 8080. The page polls `/api/state` every 250ms with the Fetch API, receives JSON, and updates progress bars via the DOM. When `pipeline_done` is true the page posts to `/api/shutdown`, which triggers the server thread to call `play_video()` and then `os._exit(0)`.

6. Compliance Analysis: Code vs. Proposal

6.1 Mandatory Objectives

- **Requirement:** Multi-process pipeline using `fork()` with orchestrator managing N workers via `waitpid()`

Status: FULLY MET -- `orchestrator.c` implements `spawn_worker()` with `fork()+execl()` and `monitor_workers()` with `waitpid(WNOHANG)` in a polling loop. The orchestrator is the parent; workers are distinct child processes with separate PIDs.

- **Requirement:** Spawn pthreads within each worker for frame-level parallel encoding

Status: FULLY MET -- `worker.c` spawns `NUM_THREADS = 4` pthreads per worker in `run_pthreads_and_encode()`. Threads operate concurrently on the frame index pool and join before the worker exits.

- **Requirement:** Protect shared output buffer with `pthread_mutex_t` to prevent data races
Status: FULLY MET -- `buffer_mutex` guards all writes to `output_buffer[]` and `buffer_index`. `work_mutex` guards `current_frame_idx`. Both are statically initialized and never destroyed (which is acceptable for process-local mutexes that live for the entire process lifetime).
- **Requirement:** Use POSIX named semaphores to coordinate completion between pipeline stages
Status: FULLY MET -- N named semaphores (one per worker) are created by the orchestrator with initial value 0. Each worker calls `sem_post()` exactly once when done. The orchestrator calls `sem_timedwait()` for each. The 60-second timeout prevents deadlock on worker failure.

6.2 Additional Objectives

- **Requirement:** Multiple encoding profiles (High Quality, Fast, Balanced)
Status: NOT IMPLEMENTED -- The FFmpeg command in `worker.c` uses a hardcoded profile: `-c:v libx264 -preset fast -crf 28`. There is no mechanism for the user to select a profile, and no alternative preset or CRF values are defined.
- **Requirement:** Load balancer distributing frame batches based on worker utilization
Status: NOT IMPLEMENTED -- Frame distribution in `main()` uses simple integer division: `base = total_frames / nw`, `extra = total_frames % nw`. This is a static round-robin partition, not a dynamic load balancer. There is no feedback loop that would reassign frames from a slow worker to a faster one.
- **Requirement:** Performance metrics: throughput, per-stage latency, CPU utilization
Status: PARTIALLY MET -- `worker_frames[]` in shared memory gives a live frames-encoded count visible to the GUI, which can be used to derive throughput. However, there are no per-stage latency measurements, no CPU utilization readings, and no throughput graph in the GUI. The GUI displays a progress percentage but not frames-per-second.
- **Requirement:** Error recovery: detect and respawn failed workers without halting the pipeline
Status: FULLY MET -- `monitor_workers()` calls `waitpid(WNOHANG)` every 100ms. On a non-zero exit it logs the failure and calls `spawn_worker()` again, up to `RESPAWN_MAX = 3` times per slot. `worker_frames[id]` is reset to 0 on respawn. The pipeline only marks a slot as permanently failed after three failed attempts.
- **Requirement:** Structured log file with timestamped events

Status: NOT IMPLEMENTED -- The Makefile redirects orchestrator stdout to orchestrator.log via shell redirection, but the code itself writes no timestamps and no structured format. Log entries are plain printf lines without dates, times, or severity levels.

7. Architectural Deviations from the Proposal

Several design choices in the implementation differ from what the proposal described. These are not bugs but conscious deviations, some of which improve the system.

- **Semaphore semantics changed from stage-based to worker-based:** The proposal described three named semaphores (`sem_stage1_done`, `sem_stage2_done`, `sem_pipeline_complete`) to coordinate a Read -> Encode -> Write three-stage pipeline. The implementation instead creates one semaphore per worker (`/vidpipe_sem_0` through `/vidpipe_sem_N-1`). This is simpler and more scalable: the proposal's three-stage model assumes a fixed pipeline topology, while the per-worker model works for any number of workers. The downside is that the Read and Write stages are not separately observable.

- **No PNG frame extraction step:** The proposal and the comment block at the top of `orchestrator.c` both describe step 1 as splitting the video into PNG frame files in a `frames/` directory. The implementation skips this step entirely. Workers transcode directly from the source video using FFmpeg's select filter to pick their frame range. This is more efficient (avoids gigabytes of PNG files) but means the `frames/` directory is never populated.

- **GUI uses a web browser instead of Python tkinter/ncurses:** The proposal specified Python tkinter with a fallback to ncurses for headless environments. The implementation uses Python's built-in HTTPServer to serve a Glassmorphism HTML dashboard on `localhost:8080` and opens it in the default browser with `webbrowser.open()`. This is visually richer and avoids the tkinter dependency but requires a browser and does not work headless.

- **shmget vs shm_open:** The proposal's technology stack table lists `shmget` as the IPC mechanism. `shmget` is the older System V shared memory API. The implementation uses the POSIX API (`shm_open` + `mmap`) which is newer, uses file-like named objects, and is better suited to mixing C and Python because Python's `mmap` module can directly open a file descriptor.

- **Thread simulation vs real parallel encoding:** The proposal implies that pthreads perform actual parallel encoding. In the implementation, FFmpeg (a single sequential process) does the real encoding, and the four pthreads only simulate parallel work with `nanosleep()` calls. This is an honest design compromise: FFmpeg itself is not

thread-safe to call in parallel from within the same process. The threads do still exercise mutex and shared-buffer logic as required.

8. Build and Run Instructions (Ubuntu)

Prerequisites:

- gcc with C11 support (sudo apt install build-essential)
- FFmpeg and ffprobe (sudo apt install ffmpeg)
- Python 3 standard library only -- no pip packages needed for gui_monitor.py
- mpv or vlc media player for automatic playback (sudo apt install mpv)

Build:

```
cd /path/to/project
make          # compiles orchestrator and worker binaries
```

Run with GUI monitor (recommended):

```
make run WORKERS=4    # launches orchestrator + opens browser
dashboard
```

Or manually in two terminals:

```
Terminal 1: python3 gui_monitor.py
Terminal 2: ./orchestrator input.mp4 4 > orchestrator.log 2>&1
```

Clean up:

```
make clean    # removes binaries, segments/, frames/,
output_final.mp4
```

Note: If the orchestrator is killed without Ctrl+C cleanup, stale IPC objects may remain.

Remove them manually with:

```
rm -f /dev/shm/vidpipe_shm
ls /dev/shm/vidpipe_sem_*  && rm /dev/shm/vidpipe_sem_*
```

9. Conclusion

The Multi-Process Video Encoding System successfully implements all four mandatory OS objectives from the project proposal. The codebase demonstrates fork-exec process creation, POSIX thread parallelism, mutex-protected critical sections, and named semaphore synchronization in a single cohesive project that produces tangible output (a re-encoded video file).

Three of the five additional objectives are not implemented: multiple encoding profiles, a utilization-based load balancer, and a timestamped structured log file. The performance metrics objective is partially met. These gaps represent straightforward extensions that could be added without restructuring the existing code.

The main architectural deviation from the proposal -- using per-worker semaphores instead of per-stage semaphores -- is a reasonable improvement for an N-worker system. The GUI technology change from tkinter to a web dashboard is also a net improvement in usability, at the cost of losing the headless ncurses fallback.

Overall the project is a solid demonstration of POSIX systems programming and OS concurrency concepts. Every IPC primitive taught in the course is present in the code: shared memory provides the high-bandwidth state channel, semaphores provide the low-level completion signal, mutexes protect in-process shared data structures, and process monitoring with waitpid provides fault tolerance.